

7 September 2026 · 9 min read

Notes from building a CQRS service

The architecture decisions I would defend in review, and the three defects that only surfaced once I stopped reading the code and started running it.

Aarti Hariharno · Full Stack Software Engineer

[aarti31h/subscription-service](https://github.com/aarti31h/subscription-service) ➤ (https://github.com/aarti31h/subscription-service)

.NET 8

CQRS

MediatR

EF Core

Redis

Testing

Why build this at all

Most of what I have built in five years lives in private client repositories. That is normal for agency and product work, and it leaves a gap: I can describe how I structure a service, but I cannot hand anyone the code and let them judge for themselves.

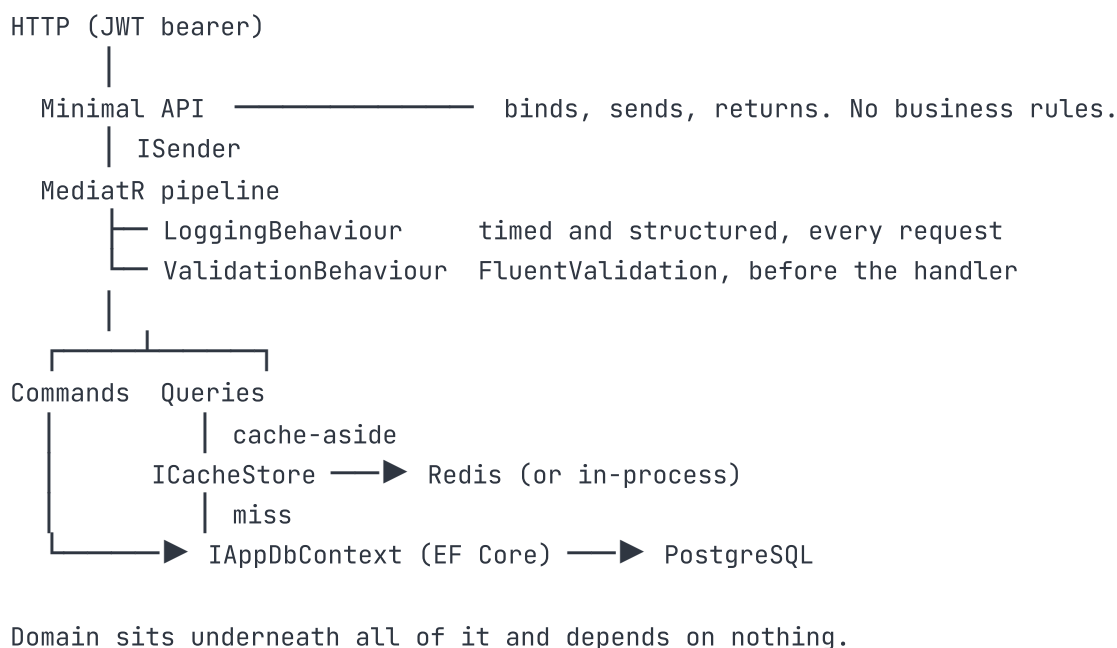
So I extracted the shape of a system I have shipped several times — a finance service with a transactional write path and a hot read path — into something small enough to read in one sitting and complete enough to run. Subscription billing turned out to be a good miniature: writes are rule-heavy and transactional, reads are frequent and tolerant of being slightly stale, and confusing the two is how an API ends up slow and incorrect at the same time.

What follows is the part I actually care about: not that it works, but why it is arranged the way it is, and what happened when I ran it.

The shape of it

Four projects, with dependencies pointing inwards. The domain references nothing. The application layer references only the domain and reaches the outside world through three interfaces it owns — persistence, caching and time. Infrastructure implements those. The API wires them together.

REQUEST PATH



The practical payoff of that arrangement is small but real: the application layer has no PostgreSQL or Redis package reference at all, so its tests need neither. That is not architectural purity for its own sake — it is the difference between a test suite that runs in two seconds and one that needs Docker.

Decisions I would defend in review

CQRS, but only as far as it pays

Creating a subscription validates input, checks an invariant that spans rows, and writes in a transaction. Reading one returns a flat projection that is fetched far more often than it changes. Pushing both through a single service class means one model serving two masters: either the read carries change-tracking overhead it does not need, or the write is bent into a shape chosen for reads.

Separating them let me put AsNoTracking and a cache on the read path without touching write correctness. Worth being precise about the limit, though: this is CQRS as a code separation, not separate read and write databases. That further step buys eventual-consistency problems, and this domain does not have a scaling reason to pay for them. Naming a pattern is not the same as needing all of it.

The cache is invalidated on write, not left to expire

A five-minute TTL sounds harmless until you follow it through. A user cancels a subscription, the confirmation succeeds, and the next screen still shows Active. Nothing has errored. Nothing is in the logs. They contact support, and support cannot reproduce it because by then the key has expired.

So writes drop both the item key and the owning list key, and the TTL exists only as a safety net for anything that changes data outside these handlers. The keys themselves live in one internal class, because scattered string interpolation is exactly how an invalidation quietly stops matching the key it was meant to clear.

The cache is allowed to fail

Every Redis operation is wrapped and falls through to the database on failure. A cache is an optimisation, not a source of truth: if Redis is unreachable the correct behaviour is to be slow, not to return an error. That is only defensible because each failure is logged at warning level and the caller has a working fallback — swallowing an exception with no fallback is a different thing entirely, and a much worse one.

State transitions belong to the entity

“A cancelled subscription cannot be cancelled again” is a property of a subscription, not of the endpoint that happens to call it. Put that check in the handler and the next caller — a background job, a message consumer, an admin tool — has to remember it independently. `Subscription.Cancel` throws, and every path gets the rule for free.

One rule deliberately did not go there. “No duplicate active subscription per plan” spans rows, and an aggregate cannot see its siblings. That one lives in the handler, and the asymmetry is intentional rather than an oversight.

Three defects, and what caught each

This is the part I would want to read in someone else's write-up. The code compiled and the unit tests were green well before any of these appeared. Each one needed a different kind of verification to surface.

1. The content type that lied

CAUGHT BY

An integration test asserting on the response content type, not just the status code.

Error responses are RFC 7807 problem details, produced by one middleware. It set the status code, set `Response.ContentType` to `application/problem+json`, then wrote the body with `WriteAsJsonAsync`.

`WriteAsJsonAsync` overwrites the content type. Silently. Every error response was going out as `application/json`.

THE FIX

```
// Wrong – the content type set here is discarded.
context.Response.ContentType = "application/problem+json";
await context.Response.WriteAsJsonAsync(problem);

// Right – pass it to the call that owns the response body.
await context.Response.WriteAsJsonAsync(
    problem, options: null, contentType: "application/problem+json");
```

What makes this worth writing down is how invisible it was. The status codes were correct, so every status-code assertion passed and a browser showed exactly the right thing. Only a client branching on the content type would have been wrong — and it would have been wrong quietly, in production, in someone else's codebase. The test that caught it was one extra line asserting on a header.

2. The crash on the path I had advertised

CAUGHT BY

Running the built container, rather than trusting that a green build meant a working service.

Every route sits behind `RequireAuthorization`. To keep the service runnable on a clean machine, I made the JWT registration conditional: no signing key configured, no authentication services registered. `UseAuthentication()` was still called unconditionally.

The container started, failed to resolve `IAuthenticationSchemeProvider`, and died. The README I had just written told people to clone and run it.

The interesting part was not the null-check mistake, it was realising the design was wrong underneath it. An API where every route requires authorisation cannot meaningfully run without auth configured. There were only two honest options, and “quietly disable authentication” was not one of them — that turns a forgotten environment variable into a publicly readable API with nothing in the logs to suggest anything is wrong.

So it now fails fast outside Development with a message naming the setting, and in Development it generates an ephemeral key and says so on stdout. Clone-and-run still works, no secret is committed, and a misconfigured deployment fails loudly instead of silently opening the door.

3. The bug that only existed on someone else's machine

CAUGHT BY

CI, running on a different SDK patch than my laptop.

The Redis read deserialised a RedisValue directly. RedisValue converts implicitly to both string and ReadOnlySpan<byte>, so the call is ambiguous — but only under a compiler strict enough to say so. My local SDK accepted it. CI did not.

The fix was one explicit cast. The lesson was that “it builds locally” had quietly become a claim about my machine rather than about the code, so I added a global.json pinning the SDK band. Local and CI now have to agree, or they tell me.

What this is not

Being straight about the limits is part of the point.

- The integration tests run against the EF Core in-memory provider, not PostgreSQL. That keeps them fast and dependency-free in CI, but it verifies no provider-specific SQL, no index usage and no database constraints. Anything relying on a real query plan still needs a container-backed test.
- EnsureCreated is used in Development instead of migrations. That is a convenience, and it is called out as one in the code.
- There is no message bus here. Publishing a SubscriptionCancelled event is the obvious next step, and it is listed as such rather than pretended into existence.

A reference implementation that claims to be production-ready is worth less than one that says exactly where it stops. The second kind survives the follow-up question.

What I took from it

None of the three defects were found by reading the code, and none were found by the unit tests — which were green throughout. One needed an assertion on something I had assumed rather than checked. One needed the artefact to actually be run. One needed a machine that was not mine.

A green build tells you the code compiles and the things you thought to check still hold. It does not tell you the service starts.

That is the habit I would bring to a team, more than any particular opinion about CQRS: verify the thing you are actually shipping, on a machine that is not yours, and assert on the parts you assumed rather than only the parts you wrote.

The full service — architecture, decisions, tests and all three fixes — is public, and the README explains each trade-off it accepted.

READ THE CODE

**aarti31h/subscription-
service**

↗ (https://github.com/aarti31h/subscription-
service)

Let's connect